

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Построение и анализ алгоритмов»
ТЕМА: РАССТОЯНИЕ ЛЕВЕНШТЕЙНА

Студент гр. 4345

Пикалев Н.Г.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы.

Изучить и реализовать алгоритм поиска расстояния Левенштейна.

Задание.

Расстоянием Левенштейна назовём минимальное количество операций вставки одного символа, удаления одного символа и замены одного символа на другой, необходимых для превращения одной строки в другую.

Разработайте программу, осуществляющую поиск расстояния Левенштейна между двумя строками.

Пример:

Для строк `pedestal` и `stien` расстояние Левенштейна равно 7:

- Сначала нужно совершить четыре операции удаления символа: `pedestal` $\rightarrow$ `stal`.
- Затем необходимо заменить два последних символа: `stal` $\rightarrow$ `stie`.
- Потом нужно добавить символ в конец строки: `stie` $\rightarrow$ `stien`.

Параметры входных данных:

Первая строка входных данных содержит строку из строчных латинских букв. ($S, 1 \leq |S| \leq 2550$).

Вторая строка входных данных содержит строку из строчных латинских букв. ($T, 1 \leq |T| \leq 2550$).

Параметры выходных данных:

Одно число L , равное расстоянию Левенштейна между строками S и T .

Вариант 3а.

Добавляется 4-я операция со своей стоимостью: последовательная вставка двух одинаковых символов.

Основные теоретические положения.

Расстояние Левенштейна, или редакционное расстояние, - метрика сходства между двумя строковыми последовательностями. Расстояние соответствует минимальному числу односимвольных преобразований (удаления, вставки или замены), необходимых, чтобы превратить одну последовательность в другую.

Расстояние Левенштейна активно используется для исправления ошибок в словах, поиска дубликатов текстов, сравнения геномов и прочих полезных операций с символьными последовательностями.

Вычислить расстояние Левенштейна между двумя последовательностями можно с помощью метода Вагнера-Фишера. Необходимо составить матрицу D и каждый её элемент вычислить по рекуррентной формуле (см. Рис. 1):

$$D(i, j) = \begin{cases} 0, & i = 0, j = 0 \\ i, & j = 0, i > 0 \\ j, & i = 0, j > 0 \\ \min\{ \\ \quad D(i, j - 1) + 1, \\ \quad D(i - 1, j) + 1, \\ \quad D(i - 1, j - 1) + m(S_1[i], S_2[j]) \\ \} & j > 0, i > 0 \end{cases}$$

Рисунок 1 – Метод Вагнера-Фишера

Выполнение работы.

Для реализации алгоритма поиска расстояния Левенштейна с помощью матрицы размером $N * M$ потребуются затраты по времени и памяти, равные $O(N * M)$. При работе с длинными последовательностями время выполнения может оказаться недопустимым. Оптимизировать алгоритм можно путём сохранения лишь двух строк матрицы, так как для вычисления любой строки матрицы требуется лишь предыдущая строка. После оптимизации алгоритма затраты по памяти будут составлять $O(\min(N, M))$, затраты по времени останутся равными $O(N * M)$.

Первая строка матрицы будет заполнена числами от 0 до N , первый элемент каждой строки будет равняться индексу текущего элемента в первой поданной на вход программе строке. Остальные числа будут заполняться согласно формуле $\min(\text{add}, \text{delete}, \text{replace})$.

$\text{replace_val} = \text{prev}[j - 1] + \text{cost_replace}$

$\text{delete_val} = \text{prev}[j] + \text{cost_delete}$

$\text{insert_val} = \text{curr}[j - 1] + \text{cost_insert}$

Решение задачи было дополнено согласно варианту 3а. В варианте 3а добавляется 4-я операция со своей стоимостью, которая позволяет последовательно вставить два одинаковых символа. Операция применяется только тогда, когда есть возможность вставить два одинаковых символа. Если вставить два одинаковых символа не представляется возможным, то операция не применяется.

Затраты алгоритма по времени: $O(N * M)$

Затраты алгоритма по памяти: $O(\min(N, M))$

Исходный код программы см. в приложении А.

Результаты тестирования программы см. в таблице 1.

Таблица 1 – Тестирование программы

Ввод	Результат	Комментарий
pedestal stien	7	Пример из условия задачи
qwerty qwerty	0	Два одинаковых слова
yihktjkggghglggkghhhjj elfkglhhlyhlyh	17	Ещё один пример
www waawaaw	2	Пример 4-ой операции

Выводы.

В ходе выполнения данной лабораторной работы был изучен и реализован алгоритм поиска расстояния Левенштейна для двух последовательностей с использованием метода динамического программирования. Были закреплены навыки работы с матрицами, использование памяти оптимизировано до $O(\min(N, m))$. Классический алгоритм был дополнен согласно условию варианта 3а.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: lb3.py

```
cost_replace, cost_insert, cost_delete, cost_insert2 = map(int,
input("Стоимость операций: ").split())

S = input("Первая строка: ").strip()
T = input("Вторая строка: ").strip()

lenS, lenT = len(S), len(T)

prev = [j * cost_insert for j in range(lenT + 1)]

for i in range(1, lenS + 1):
    curr = [i * cost_delete] + [0] * lenT

    for j in range(1, lenT + 1):
        print(f"Символ из первой строки: {S[i-1]}")
        print(f"Символ из второй строки: {T[j-1]}")

        if S[i-1] == T[j-1]:
            print(f"Символы совпадают.")
            curr[j] = prev[j-1]

        else:
            replace_val = prev[j-1] + cost_replace
            delete_val = prev[j] + cost_delete
            insert_val = curr[j-1] + cost_insert
            insert2_val = float('inf')

            print(f"Символы не совпадают.")
            print(f"Стоимость замены: {replace_val}")
            print(f"Стоимость удаления: {delete_val}")
            print(f"Стоимость вставки: {insert_val}")

            if j >= 2 and T[j-2] == T[j-1]:
                insert2_val = curr[j-2] + cost_insert2
                print(f"Стоимость                двойной                вставки:
{insert2_val}")

            else:
                print(f"Двойная вставка невозможна")

            curr[j] = min(replace_val, delete_val, insert_val,
insert2_val)
            print(f"Минимальная стоимость: {curr[j]}")

    print(f"Текущая строка: {curr}\n")

    prev = curr

print(f"Расстояние Левенштейна: {prev[lenT]}")
```